

CSCE 650 VIRTUAL REALITY · MODULE 06 · LECTURE NOTES

Latency

The central problem, and everything built to hide it.

MODULE OUTCOMES

Account for every stage of the motion-to-photon budget.

State the latency threshold and what exceeding it does.

Explain timewarp and what it can and cannot correct.

Explain spacewarp and its artefacts.

Measure latency rather than estimating it.

1 The motion-to-photon budget

Stage	Typical cost	Under your control?
Sensor sampling and readout	~1 ms	No.
Fusion, filtering, pose computation	~1 ms	No.
Application CPU — simulation, culling, draw submission	2–11 ms	Yes. This is yours.
Application GPU — rendering both eyes	2–11 ms	Yes. This is yours.
Compositor — timewarp and distortion	1–2 ms	No.
Scan-out — writing the image to the panel	Up to a full refresh period	Only by increasing the refresh rate.
Pixel response — the panel physically changing state	1–3 ms	Panel technology; OLED is faster than LCD.

The target is under 20 ms end to end. Notice how much of the budget the application does not control — which is precisely why the portion it does control must be managed ruthlessly.

20 MS IS A THRESHOLD, NOT A PREFERENCE

Below roughly 20 ms motion-to-photon, the virtual world feels attached to the room. You turn your head and the world stays put, as the real world does.

Above it, the world feels attached to your head and dragged along behind it. The sensation is distinctive once you have experienced both.

Controlled experiments find people detecting latency differences down to a few milliseconds — far below the level at which they can say what is wrong. Sensitivity substantially exceeds the ability to describe.

And the symptom is not a perception of lag. Users do not report 'the image is late'; they report feeling unwell, disoriented, or simply that the experience is unpleasant. **The latency is converted into discomfort** by the mechanism in Module 07, which is why it is a physiological constraint rather than a quality setting.

This one number drove the entire architecture of modern VR: low persistence displays, timewarp, pose prediction, the separate compositor process, and the high refresh rates all exist to get under it.

2 Timewarp

REPROJECT THE FINISHED FRAME TO A NEWER POSE

The application renders from a pose that was predicted some milliseconds earlier. **Just before scan-out, the compositor takes a fresh pose — much newer — and reprojects the completed image to match it.**

For rotation this is nearly exact. Rotating the view of a scene from the same viewpoint is a pure image-space warp: every pixel that was visible before is still visible, merely in a different place on the display. No information is missing.

The reduction in effective latency is large. The user's head rotation is reflected on screen with only the compositor's own latency — a millisecond or two — rather than the full pipeline's.

This is why head rotation feels solid even when an application is struggling, and since rotation is by far the dominant component of head motion, timewarp covers most of the problem. It is the single most important technique in VR engineering.

	Rotation	Translation
What changes	The view direction, from the same point.	The viewpoint itself moves.
Information needed	None beyond the rendered image. Every visible surface is still visible.	Geometry that was occluded before and is visible now — which was never rendered and does not exist in the frame.
Quality	Nearly exact.	Approximate at best. Positional timewarp uses the depth buffer to reproject, and leaves disocclusion holes that must be filled by guessing.

	Rotation	Translation
Edge handling	The field edges may lack data, which is why the render target is slightly oversized.	Holes appear in the interior of the image, not only at the edges.

This asymmetry is the key limitation. It means translation latency is intrinsically harder to hide than rotation latency — and it is one reason that applications involving a lot of positional head movement, such as anything requiring leaning or crouching, are more demanding than they appear.

ASYNCHRONOUS TIMEWARP IS A SAFETY NET

ATW runs on its own schedule, in a high-priority GPU context that can pre-empt the application, and reprojects whatever the most recent *complete* frame is — whether or not the application has finished a new one.

So a missed frame does not mean a repeated frame. The previous frame is re-warped to the current head pose, and head rotation remains correct and smooth even though the content is one frame old.

The world stays stable; moving objects stutter. That is a far better failure mode than the entire world juddering, because it is the world's stability relative to the head that the vestibular system cares about.

It is a safety net and not a licence. An application that relies on ATW to reach its presentation rate is effectively running its animation and simulation at half rate, which users notice as stutter in everything that moves — and it leaves no headroom for the occasional genuinely expensive frame.

3 Spacewarp

SYNTHESISING FRAMES, AND WHAT IT COSTS

Asynchronous spacewarp goes further than timewarp: it generates an entirely new frame by extrapolating the motion of objects within the scene, using motion vectors and the depth buffer from previous frames.

So an application rendering at 45 Hz can present at 90 Hz, with every alternate frame synthesised rather than rendered.

The artefacts are characteristic and recognisable once you know them: wobbling or rippling at the edges of moving objects; smearing behind fast-moving things; and **disocclusion holes** where background is revealed that was never rendered, filled by inpainting that is sometimes obviously wrong.

Supply accurate motion vectors and depth if your engine can. The synthesis quality depends heavily on them, and an engine that provides good motion vectors gets markedly better results. **And as with ATW, treat it as a net rather than a plan** — an application designed around spacewarp is an application with permanent artefacts.

- **Hit the frame budget.** Everything else in this list is secondary, because a missed frame costs an entire refresh period — more than every other optimisation combined.
- **Late latching:** read the head pose as late as possible in the frame, after CPU work and immediately before the GPU needs it (Module 05). This shortens the prediction horizon and therefore the error.
- **Shorten the CPU–GPU pipeline.** Flat-screen engines buffer two or three frames deep to maximise throughput, and **each buffered frame is a full refresh period of added latency**. VR wants a shallow pipeline even at some cost in average throughput. **This surprises people porting an existing engine,** because it trades away a tuning decision that was correct in the original context.
- **Avoid temporal effects requiring several frames of history.** They add perceived latency even when frame timing is correct, because the image lags the pose by more than one frame (Module 02 §3).
- **A higher refresh rate reduces scan-out latency directly,** which is the one stage outside your control that responds to anything.

MEASURE LATENCY; DO NOT ESTIMATE IT

The pendulum or high-speed-camera method: swing a tracked controller in front of a camera positioned to see both the physical controller and the headset display simultaneously, record at a high frame rate, and count the frames between the physical motion and the displayed response. Crude, cheap, and it measures the whole pipeline including everything you cannot instrument.

A photodiode taped to the display, triggered against an instrumented input event, gives a precise electrical measurement and is the method used in published comparisons.

Some runtimes report an estimate. Use it for iteration, and verify it against a physical measurement at least once — runtime figures frequently omit scan-out and pixel response.

Estimating latency from frame time is wrong and badly so. Frame time omits sensor sampling, the compositor, scan-out, and pixel response — which together are usually as large as the rendering itself. **The frame-time estimate is typically about half the true figure,** which is exactly the kind of error that leads someone to conclude they are comfortably within budget when they are not.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Abresh — Latency: the sine qua non of AR and VR (free)	Required reading for this module. The clearest account of why latency matters and what the threshold means.
LaValle — Virtual Reality, Chapter 7.4 (free)	Timewarp, prediction, and the latency budget, with the mathematics.
Van Waveren — The Asynchronous Time Warp for Virtual Reality on Consumer Hardware (free)	The ATW paper of §2, including the GPU pre-emption problem that made it hard.
Meta — Asynchronous Spacewarp documentation (free)	What spacewarp needs from your application, and the artefacts to expect.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Account for your own application's latency budget stage by stage, measuring what you can and citing typical figures for the rest.

- 2 **Measure motion-to-photon latency physically**, with a high-speed camera or a phone's slow-motion mode. Compare against your estimate from frame time.
- 3 Deliberately add 20 ms of artificial latency and describe the experience. Then 40 ms. Stop when uncomfortable.
- 4 Disable timewarp if your runtime allows it, and compare head rotation with and without.
- 5 Construct a scene with strong parallax and translate your head laterally, looking for the limits of positional timewarp.
- 6 Deliberately miss the frame budget — add GPU work until you drop to half rate — and observe ATW keeping the world stable while moving objects stutter.
- 7 Trigger spacewarp and photograph the disocclusion artefacts behind a fast-moving object.
- 8 Reduce your engine's pipeline depth from three frames to one and measure the latency change and the throughput cost.
- 9 Compare measured latency at 72 Hz and 90 Hz on the same application.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 List the seven stages of the motion-to-photon pipeline and say which you control.
- 2 What is the latency threshold, and what is the symptom of exceeding it?
- 3 Explain timewarp in one sentence and say why rotation is nearly exact.
- 4 Why is translation handled badly, and what does that imply?
- 5 What does asynchronous timewarp guarantee, and why is relying on it a mistake?
- 6 What does spacewarp do, and name its three characteristic artefacts.
- 7 Give five things that actually reduce latency.
- 8 Why does pipeline depth matter more in VR than on a flat screen?
- 9 Why is estimating latency from frame time wrong, and by roughly how much?